Machine Learning Concepts

Car Price Prediction

Ruchi Suresh Rathod
24801180@stu.mmu.ac.uk

Abstract— The automotive industry is a dynamic and data-rich domain where pricing plays a pivotal role in both sales and purchases. Car prices are influenced by a variety of factors, including technical specifications, condition, age, brand reputation, and regional market trends. Understanding these factors and their interplay is essential for accurately estimating a car's value. This dataset belongs to the domain of **automotive sales and valuation**, specifically focusing on used and new cars. By leveraging the data, we aim to bridge the gap between market trends and individual vehicle characteristics for fair and informed pricing.

I. Introduction

The dataset "Adverts" contains detailed information about cars, including specifications like mileage, make, model, vehicle condition, year of registration, price, body type, fuel type, region, and colour. These features encompass both numerical and categorical data, capturing essential details about the cars and their characteristics. The primary objective is to use this data to predict car prices based on the given parameters. By analysing relationships between features such as vehicle condition, age, mileage, and brand, we aim to build a model that provides accurate price predictions.

II. Data/Domain Understanding and Exploration

1.1. Meaning of the Features

Describing the columns in dataset:

1. **public_reference:** A unique number used to identify each car listed in the advert dataset for reference. The column values are denoted as integer e.g. 202007020778260 etc.

2. **mileage:** The distance a car is driven annually, measured in miles. It helps buyers understand how much a car has been used and how it might affect its price. The column values are denoted as float number e.g. 7800.0 etc.

3. **reg_code:** A registration code/reg_code indicates the year the car was registered. This is often used by buyers to learn more about the car's history, we will be using the https://en.wikipedia.org/wiki/Vehicle_registration_plates_of_the_United_Kingdom link for understanding the relation between year of registration and registration code respectively. The values in this column are denoted as object e.g. 61, C, F etc.

4. **standard_colour:** The colour of the car's exterior, such as black, white, red, or blue. This helps buyers pick cars in their preferred colour. The column values are denoted as object e.g. 'Blue' 'Brown' etc.

5. **standard_make:** The brand or manufacturer of the car. The column values are denoted as object e.g. 'Jaguar' 'SKODA' 'Vauxhall'.

6. **standard_model:** The specific name or type of car within a brand. This distinguishes different designs and features offered by the same manufacturer. The column values are denoted as object e.g.'XC90' 'XF' 'Yeti' 'Mokka'.

7. **vehicle_condition:** Describes whether the car is new or used. This gives a sense of the car's wear and tear. The column values are denoted as object e.g. 'NEW' 'USED'.

8. **year_of_registration:** The year when the car was officially registered for use. This helps estimate the car's age, which can influence its price and condition. The column values are denoted as float e.g. 2016, 2005 etc.

9. **price:** The listed cost of the car in pounds sterling (£) UK currency. This is the amount a buyer would need to pay to purchase the vehicle. The column values are denoted as integer e.g. 73970 etc.

10. **body_type:** The style or design of the car's body, such as sedan, SUV, hatchback, or convertible. This helps buyers match their preferences for space, utility, or aesthetics. The column values are denoted as object e.g. 'SUV' 'Saloon'.

11. **crossover_car_and_van:** Indicates if the car is a crossover (a mix of a car and an SUV) or a van. This column answers whether the car has a dual-purpose design, marked as either "true" (yes) or "false" (no). The column values are denoted as boolean e.g. True/False

12. **fuel_type:** Specifies the type of fuel or energy the car uses to operate, such as petrol, diesel, electricity, or hybrid. This is essential for buyers considering running costs and environmental impact. The column values are denoted as object e.g. 'Petrol Plug-in Hybrid' 'Diesel'.

First, we import required libraries such as - Pandas for data analysis, NumPy for numerical manipulation, Matplotlib and Seaborn for visualization and Warnings to suppress different warnings for deprecated methods and functions. Followed by loading and reading the dataset and then checking the loaded data. Once the data is loaded, we investigate the data types, unique values in each column. The purpose of doing this is to glance over the dataset so we can further plan required data manipulation and management for detailed analysis.

```
# read data
df = pd.read_csv('adverts.csv')
```

```
df.head(2)
```

	public_reference	mileage	reg_code	standard_colour	standard_make	standard_model	vehicle_condition	year_of_registration	price	body_type	cross
0	202006039777689	0.0	NaN	Grey	Volvo	XC90	NEW	NaN	73970	SUV	
1	202007020778260	108230.0	61	Blue	Jaguar	XF	USED	2011.0	7000	Saloon	

Exploring further for better understanding.

```
def get_unique(dataframe, column_name):
    col = dataframe[column_name][:5]
    check_unique = print(f"The unique values in column {column_name} are {col.unique()}")
    return check_unique
```

```
reg_code_unique = get_unique(df, 'reg_code')
colour_unique = get_unique(df, 'standard_colour')
make_unique = get_unique(df, 'standard_make')
model_unique = get_unique(df, 'standard_model')
vehicle_condition_unique = get_unique(df, 'vehicle_condition')
body_type_unique = get_unique(df, 'body_type')
fuel_type_unique = get_unique(df, 'fuel_type')
```

We have created a function get_unique to retrieve unique values of each column by slicing [:5] to give first 5 unique ones only, in order to understand what kind of values we will be dealing with in further process.

Beginning with data wrangling, we have created a copy of the original dataset, on which we will be doing all the operations without affecting the original data and keeping it unaltered. This enable us in safe experimentation and preserving reproducibility. Now we can have a look at the data, understand it and move forward step by step for manipulation using the copy of original dataset.

```
car_df['public_reference'].nunique()
```

```
402005
```

Here, we observe that the public reference number contains 402,005 unique values, matching the number of rows in our dataset (402,005 x 12). This indicates that the public reference column does not influence car prices as there is no unique-ness to this dataset feature. Therefore, we have decided to drop it in further data cleaning process.

Next, we segregate our data into numerical and categorical columns and assign it to respective variables. This will help us in optimizing our exploration and perform more targeted analysis.

Types of Features

1. Numeric Features: 'mileage', 'year_of_registration', 'price'.

2. Categorical Features: 'reg_code', 'standard_colour', 'standard_make', 'standard_model', 'vehicle_condition', 'body_type', 'crossover_car_and_van', 'fuel_type'.

```
num_df.describe()   # describe the numeric df summary
```

	mileage	year_of_registration	price
count	401878.000000	368694.000000	4.020050e+05
mean	37743.595656	2015.006206	1.734197e+04
std	34831.724018	7.962667	4.643746e+04
min	0.000000	999.000000	1.200000e+02
25%	10481.000000	2013.000000	7.495000e+03
50%	28629.500000	2016.000000	1.260000e+04
75%	56875.750000	2018.000000	2.000000e+04
max	999999.000000	2020.000000	9.999999e+06

There are a lot of outliers and noise in dataset which we will be handling at a later stage of processing. Now, lets check the skewness of numerical data.

```
# checking the skewness
num_df.skew()
```

```
mileage                1.451132
year_of_registration   -87.909954
price                  154.681527
dtype: float64
```

Our analysis of skewness brings so much to understand here:

- **Mileage**: The **mileage** column exhibits a skewness value of 1.45, indicating a right-skewed distribution. This suggests that most cars in the dataset have higher mileage, with relatively fewer cars having low mileage. This could indicate that the dataset may be dominated by older, higher-mileage vehicles, while low-mileage vehicles are less common.
- **Year of Registration**: The column for year_of_registration shows a skewness value of -87.91, which indicates a significant left skew. This suggests that the dataset has a higher concentration of older vehicles, with relatively few newer cars. Such a distribution could point to a larger proportion of used cars in the dataset, with fewer newly registered vehicles available for sale.
- **Price**: The price column shows a skewness value of 154.68, revealing a strong right skew. This indicates that majority of cars are priced lower, with a smaller number of high-priced vehicles. This distribution suggests that the dataset likely consists of more budget-friendly cars, with high-end or luxury vehicles being less frequent.

Hence, these skewness values provide valuable insights into the nature of the dataset, highlighting that the distribution of cars is generally weighted towards older, higher-mileage, and lower-priced vehicles, which is consistent with trends seen in the used car market.

```
# Lets check the correlation of our numeric features using heatmap
sns.heatmap(num_df.corr(), annot=True)
plt.show()
```

From above, following analysis of the correlation coefficients can be interpreted:

Year of Registration vs. Price: The correlation coefficient of 0.1 between the year of registration and price shows a weak positive correlation. This means that, on average, cars with newer registration years tend to have slightly higher prices.

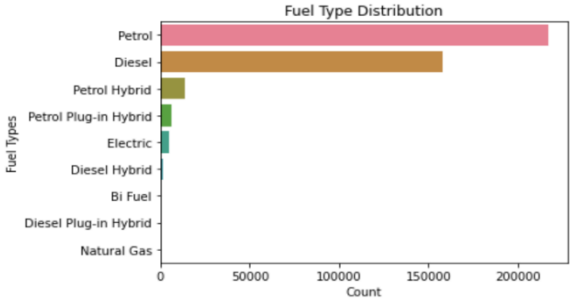
Mileage vs. Price: The correlation coefficient of -0.16 between mileage and price indicates a weak negative correlation. This suggests that as a car's mileage increases, its price tends to decrease slightly.

Year of Registration vs. Mileage: The correlation coefficient of -0.38 between the year of registration and mileage indicates a moderate negative correlation. This suggests that newer cars generally have lower mileage compared to older ones.

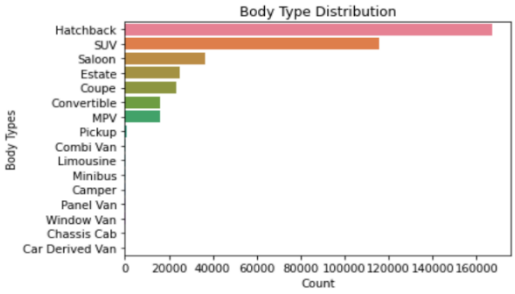
This analysis highlights how different factors are related, although the impact on car prices and mileage varies, and other variables like condition, model, and market demand are also crucial in determining the value of a car.

By checking the number of unique values of each categorical columns using our n-uniq function, we chose body_type and fuel_type for plotting because they directly impact price variations and have manageable unique values (16 and 9, respectively), making them ideal for clear visualization. Other columns like reg_code, standard_make, and standard_model have too many unique values for effective plots, while binary columns like vehicle_condition and crossover_car_and_van are better analyzed through summaries.

Plotting categorical features: We create a function which will calculate distribution proportion of each categorical feature while plotting it.



The fuel_type column distribution shows that the Petrol is the most common fuel type, accounting for 54.04% of the data. Diesel follows, making up 39.39%. Hybrid options, including Petrol Hybrid (3.39%) and Petrol Plug-in Hybrid (1.53%), have a smaller share, while Electric vehicles constitute 1.19%. Less common types include Diesel Hybrid (0.35%), Bi-Fuel (0.05%), Diesel Plug-in Hybrid (0.04%), and Natural Gas, which is nearly negligible at 0.000002%. This distribution highlights a predominant reliance on petrol and diesel, with hybrids and alternative fuels still representing a niche market.



Next, we plot Body type column. The body_type column reveals the distribution: Hatchback is the most common body type, making up 41.71% of the dataset, followed by SUVs, which account for 28.88%. Other popular types include Saloon (9.13%), Estate (6.15%), and Coupe (5.80%). Less common body types include Convertible (3.99%) and MPV (3.99%), while niche categories like Pickup (0.15%), Combi Van (0.05%), and Limousine (0.04%) have minimal representation.
Rare types such as Minibus, Camper, Panel Van, Window Van, Chassis Cab, and Car Derived Van collectively represent an extremely small fraction (less than 0.02%). This distribution indicates that hatchbacks and SUVs dominate the dataset, while specialty and commercial vehicle types are significantly underrepresented.

Insights to generate outcome: After exploring the dataset and visualizing the relevant data, we have identified several key columns that are instrumental for our analysis. These columns include mileage, standard_colour, standard_make, standard_model, vehicle_condition, year_of_registration, price, crossover_car_and_van, and fuel_type. Each of these factors plays a crucial role in understanding the dynamics of the dataset and in driving further analysis. By examining these columns, we can raise several important questions that can guide our exploration:

- **Price and its Influencing Features:** We seek to understand the relationships between price and other features. For example, how does mileage or vehicle_condition influence the price of the vehicle? Are there certain car makes or models that have a higher market value?
- **Impact of Mileage on Car Value:** Given that mileage is a critical factor in determining the wear and tear of a vehicle, we want to explore how it impacts the price and overall value of the car. Does higher mileage correlate with a lower price, or are there exceptions?
- **Average Price by Body Type:** By categorizing the vehicles based on their body type, we can examine whether certain types (like sedans, SUVs, or hatchbacks) command a higher price in the market, and identify trends within different categories.
- **Price Variations by Fuel Type:** Another key area to explore is the relationship between fuel_type (e.g., petrol, diesel, electric) and the price of the vehicle. Do fuel-efficient or eco-friendly vehicles tend to have higher prices?
- **Year of Registration and Its Effect on Price:** Lastly, understanding whether the year_of_registration has any significant impact on the average price is essential. Do newer cars fetch higher prices, or is there a certain threshold where the year of registration becomes less relevant?

Through these analyses, we aim to uncover valuable insights that can guide better decision-making for car buyers, sellers, and industry analysts. By understanding how these factors interrelate, we can create predictive models and develop a more comprehensive understanding of the market trends.

Data Cleaning (Handling Missing values, Outliers and Noise): From above exploration and data wrangling we note that we need to perform operations to achieve clean and useable data like:

- Dropping the columns not related directly to price variation - (public_reference).
- Dealing with missing values in columns (reg_code and year_of_registration).
- Dropping columns which makes least sense to our dataset - (vehicle_condition & reg_code)

- Dealing with missing values in columns (mileage, standard_colour, body_type and fuel_type).
- Deal with error values/ errant data available in dataset. (outliers and noise)

III. Data Processing for Machine Learning

a. Dealing with missing values in the dataset:

```
# checking nulls in each column
car_df.isnull().sum().sort_values(ascending=False)
```

```
year_of_registration      33311
reg_code                  31857
standard_colour            5378
body_type                   837
fuel_type                   601
mileage                     127
standard_make                 0
vehicle_condition             0
standard_model                0
price                         0
crossover_car_and_van         0
dtype: int64
```

This indicates that 'year_of_registration' and 'reg_code' nulls should be handled first.

i. Dealing with year_of_registration feature missing values: We can utilize the "Wikipedia" link provided along with dataset to understand the relationship between reg_code and year_of_registration. It can be observed from the link https://en.wikipedia.org/wiki/Vehicle_registration_plates_of_the_United_Kingdom

- from Age identifiers and suffix/prefix letter series table given, we note that reg_code and year_of_registration are related in determining age of vehicle.
- Hence, most of the null values in reg_code or year_of_registration can be filled, if either of one is present by using the relationship given in Wikipedia.

```
# Lets check the rows in the dataframe where both reg_code and year_of_registration is null
car_df[(car_df['year_of_registration'].isnull()) & (car_df['reg_code'].isnull())]
```

	mileage	reg_code	standard_colour	standard_make	standard_model	vehicle_condition	year_of_registration	price	body_type	crossover_car_and_
0	0.0	NaN	Grey	Volvo	XC90	NEW	NaN	73970	SUV	F
17	5.0	NaN	NaN	Nissan	X-Trail	NEW	NaN	27586	SUV	F
19	0.0	NaN	White	Volkswagen	T-Cross	NEW	NaN	25000	SUV	F
37	0.0	NaN	White	Fiat	Panda	NEW	NaN	13999	Hatchback	F
44	0.0	NaN	NaN	Honda	Civic	NEW	NaN	19495	Hatchback	F
...	...	...	...	...	...	...	...	...	...	
401860	10.0	NaN	Silver	Mitsubishi	Shogun Sport	NEW	NaN	31999	SUV	F
401890	5.0	NaN	Red	BMW	Z4	NEW	NaN	47910	Convertible	F
401902	10.0	NaN	White	BMW	3 Series	NEW	NaN	35023	Saloon	F
401905	0.0	NaN	Red	Land Rover	Range Rover	NEW	NaN	44995	SUV	F

- There is total 31570 rows where both reg_code and year_of_registration is null. Let's see if we can reduce this count by adding one more condition.
- From our above analysis, we confirmed that year_of_registration is NEGATIVELY SKEWED which means there are a greater number of "USED" Vehicles compared to "NEW". This way we can understand that the rows where year_of_registration is null and vehicle_condition is "USED" can be still filled for some vehicles either using mean, median or mode of column.
- But there seems to be no way of filling rows where both reg_code and year_of_registration is null, and vehicle_condition is "NEW"

```
car_df[(car_df['year_of_registration'].isnull())
       & (car_df['vehicle_condition'] == 'NEW')
       & (car_df['reg_code'].isnull())]
```

By checking this we conclude that there is no significant difference in the count (31249 rows) which is same as for vehicle condition 'NEW'. So, we drop values with this condition where reg_code, year_of_registration is null, and vehicle_condition is "NEW" (31249) by creating a subset using loc.

```
# Lets confirm this by checking how many "NEW" vehicles are there now in our dataframe car_df
car_df[car_df['vehicle_condition'] == 'NEW']
```

mileage	reg_code	standard_colour	standard_make	standard_model	vehicle_condition	year_of_registration	price

Further, we observe that we have dropped all the values that were in NEW condition. So, the numbers we have now is only for USED vehicle condition , which is of no point. Here we dropped vehicle_condition column.

```
car_df = car_df.drop(["vehicle_condition"], axis=1)
car_df.columns       #check the columns
```

```
Index(['mileage', 'reg_code', 'standard_colour', 'standard_make',
       'standard_model', 'year_of_registration', 'price', 'body_type',
       'crossover_car_and_van', 'fuel_type'],
      dtype='object')
```

Next, we check the count of the rows where both 'reg_code' and 'year_of_registration' are null in car_df. The result we get is 321 rows which cannot be filled using the Wikipedia relation, hence we drop them.

```
car_df.drop(car_df[(car_df['year_of_registration'].isnull())
                   & (car_df['reg_code'].isnull())].index, inplace=True)
```

Until now, what we have observed from dataset, the price of vehicle is related to year_of_registration and not reg_code. So, let's check how many values year_of_registration are null, which need to be filled using reg_code.

```
# checking for nulls in year_of_registration
car_df['year_of_registration'].isnull().sum()
```

```
np.int64(1741)
```

As we know, either of the one (reg_code or year_of_registration) is available in the table present in the link. we can create a dictionary using which we can fill any one of them using Wikipedia logic we found if needed. And then map accordingly.

```
group_reg_code_yr_dict = (
    car_df.groupby('reg_code')['year_of_registration'].apply(lambda x: list(set(x.dropna())))
    .to_dict())

group_reg_code_yr_dict
```

```
{'10': [1010.0, 2010.0, 2011.0, 2019.0],
 '11': [2018.0, 2011.0, 2015.0],
 '12': [2019.0, 2012.0],
 '13': [2017.0, 2018.0, 2019.0, 2020.0, 999.0, 2013.0],
 '14': [2018.0, 2019.0, 2020.0, 2014.0],
 '15': [2019.0, 2015.0],
 '16': [2016.0, 2018.0, 2019.0, 2020.0],
 '17': [2017.0, 2018.0, 2019.0, 2020.0, 1017.0],
 '18': [2018.0],
```

- Here, we see there are multiple year_of_registration for most of the reg_code which might be confusing to fill nulls.
- Our best approach to fill null values in year_of_registration should be to find the most common (mode) year for each reg_code and create a dictionary.
- Therefore, we built a logical dictionary here by mapping the reg_code values with corresponding values in year_of_registration and taking the values which has the maximum appearance (mode), to fill in the nulls.

```
mapping_dictionary = (
    car_df.groupby('reg_code')['year_of_registration']
    .apply(lambda x: x.mode().iloc[0] if not x.mode().empty else None)
    .to_dict()
)

mapping_dictionary
```

This dictionary will be very effective in filling nulls in year_of_registration, where reg_code is available.

For example: if reg_code is 64, and year_of_registration is null then using our built dictionary we see '64': 2014.0. Hence, we can fill all values in year_of_registration as 2014 where reg_code is 64. Similarly, we fill in all nulls using mapping dictionary we created above. This way we have filled majority of year of registration, but there are around 7 unique values which we couldnt fill like ['94', '85', 'CA', '723xuu', '95', '38', '37']. Note: We are not dealing with **reg_code** feature missing values because that makes least sense to our dataset. So we are dropping it.

ii. Dealing with standard_colour, body_type and fuel_type: we create a function which can handle both the cases categorical and numeric columns to impute suitable values to fill in nulls. In our function, we have an IF-Else conditional statement: For categorical, we fill using mode & For numeric, we fill using mean.

```
def groupwise_imputer(data, group_cols, target_col, is_categorical=True):
    if is_categorical:  # Categorical case: Fill with mode
        # Compute mode for each group (grouped by multiple columns)
        mode_mapping = (
            data.groupby(group_cols)[target_col]
            .agg(lambda x: x.mode().iloc[0] if not x.mode().empty else np.nan)
        )
        # Map the mode values to the missing rows
        data[target_col] = data.apply(
            lambda row: mode_mapping.loc[tuple(row[group_cols])] if pd.isnull(row[target_col]) else row[target_col],
            axis=1
        )
    else:  # Numeric case: Fill with mean
        # Compute mean for each group (grouped by multiple columns)
        mean_mapping = data.groupby(group_cols)[target_col].transform('mean')
        # Fill missing values with the group mean
        data[target_col] = data[target_col].fillna(mean_mapping)

    return data
```

Let's deal with nulls in standard_colour, body_type, fuel_type feature columns.

```
car_df.isnull().sum().sort_values(ascending=False)
```

```
standard_colour          4312
body_type                 717
fuel_type                 432
mileage                   115
standard_make               0
year_of_registration        0
standard_model              0
price                       0
crossover_car_and_van       0
dtype: int64
```

We will initially provide a list of non-null columns, such as 'standard_make' and 'standard_model' seen above, as the group_cols arguments. This approach ensures that we retrieve the most relevant mode values from the rows with non-null entries, improving the precision and accuracy of filling the missing values. After the first iteration of running the function, now left with 24 nulls in standard_colour, 35 nulls in body_type column and 16 nulls in fuel_type column.

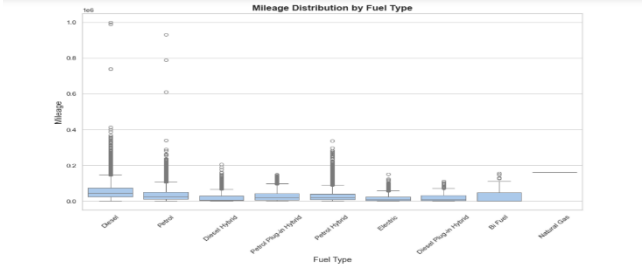
To address the remaining nulls, we proceeded by using just one column from the group_cols either 'standard_make' or 'standard_model' for filling the missing values.

iii. Dealing with Mileage

```
num_df.corr()
```

	mileage	year_of_registration	price
mileage	1.000000	-0.375541	-0.160204
year_of_registration	-0.375541	1.000000	0.102341
price	-0.160204	0.102341	1.000000

We see that from above table: The correlation heatmap shows weak relationships between mileage and other numerical columns (year_of_registration: -0.37, price: -0.25). Among these, year_of_registration has a slightly stronger link to mileage, making it a better choice for grouping during imputation. However, the weak correlations suggest that combining year_of_registration with categorical columns like standard_make or fuel_type might give better results.



We applied same function "groupwise_imputer" to mileage to fill in the nulls successfully. By this we have cleared filling missing data in all the columns.

```
# lets check if we have filled all nulls
car_df.isnull().sum().sort_values(ascending=False)
```

```
mileage                0
standard_colour        0
standard_make          0
standard_model         0
year_of_registration   0
price                  0
body_type              0
crossover_car_and_van  0
fuel_type              0
dtype: int64
```

b. Handling noise and outliers

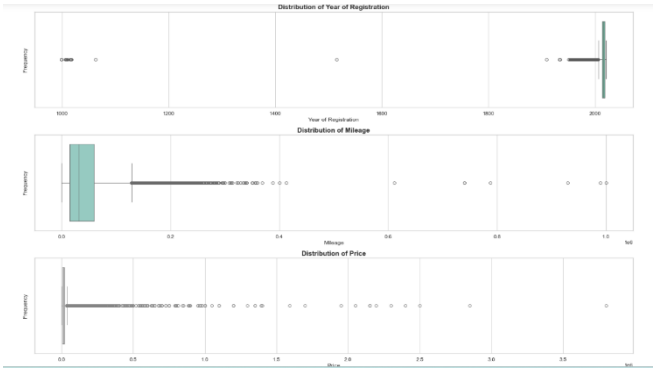
Outliers are commonly observed in numerical data, whether in continuous variables (e.g., mileage, price) or discrete variables (e.g., count data). These outliers can significantly influence the analysis and skew the overall distribution of the dataset.

	mileage	year_of_registration	price
count	370426.000000	370426.000000	3.704260e+05
mean	40929.723432	2015.010645	1.584208e+04
std	34432.050829	7.952332	2.536314e+04
min	0.000000	999.000000	1.200000e+02
25%	14453.250000	2013.000000	6.999000e+03
50%	31865.000000	2016.000000	1.189000e+04
75%	60000.000000	2018.000000	1.850000e+04
max	999999.000000	2020.000000	3.799995e+06

These may arise due to various reasons, such as measurement or data entry errors, or they may represent genuine extreme cases. Identifying and addressing these outliers is crucial, as they can distort key statistical metrics, such as the mean, median, and standard deviation. In our dataset, we observe the following potential outliers:

- An erroneous registration year of 999, extremely high and unlikely to be a realistic mileage value for most vehicles.
- A maximum mileage value of 999,999. It's well beyond any plausible year of vehicle registration and should be addressed as a data issue.

- A maximum price value of 3,799,995 which is quite high and could either reflect extreme high-end vehicles or indicate erroneous data entries.



Here, checking error values (noise) in year_of_registration, referring Wikipedia link we find that Vehicle registration plates (commonly referred to as "number plates" in British English) are the alphanumeric plates used to display the registration mark of a vehicle, and have existed in the United Kingdom since 1904. Let's filter year_of_registration lesser than 1904 as it counts to be an error data.

```
car_df[car_df['year_of_registration'] < 1904]
```

	mileage	standard_colour	standard_make	standard_model	year_of_registration	price	body_type	crossover_car_and_van	fuel_type
59010	14000.0	Blue	Toyota	Prius	1007.0	7000	Hatchback	False	Petrol Hybrid
69516	96659.0	Black	Audi	A4 Avant	1515.0	10385	Estate	False	Diesel
84501	37771.0	Black	Smart	fortwo	1063.0	4785	Coupe	False	Petrol
114737	30000.0	Red	Toyota	AYGO	1009.0	4695	Hatchback	False	Petrol
120858	27200.0	Black	MINI	Clubman	1016.0	18990	Estate	False	Diesel
190556	58470.0	Black	Fiat	Punto Evo	1010.0	3785	Hatchback	False	Petrol
199830	23000.0	Silver	MINI	Hatch	1009.0	5995	Hatchback	False	Petrol
199987	104000.0	Silver	BMW	1 Series	1008.0	4395	Convertible	False	Petrol

To deal with such year_of_registration we require reg_code, just like we previously handled. But we have dropped the reg_code column in some previous step as it was not much needed in our car_df data frame. Let's call the original data frame (df) and filter the above 1904 in it and check the result to compare year_of_registration and reg_code and also remove duplicates and consider only unique years.

```
# filtering the year_of_registration and reg_code
df_filtered = df[df['year_of_registration'] < 1904]
df_filtered[['year_of_registration', 'reg_code']]
```

	year_of_registration	reg_code
59010	1007.0	07
69516	1515.0	65
84501	1063.0	63
114737	1009.0	59
120858	1016.0	66

From above result, we can create a mapping dictionary of reg_code and year_of_registration: And then map the similar reg_code with UK vehicle registration Wikipedia link to get correct year values. Apply this dictionary to fix error data.

```
# creating a mapping dictionary:
fix_error_year_dictionary = {
    999.0:  2014.0,    #64
    999.0:  2008.0,    #08
    999.0:  2013.0,    #13
    1006.0: 2005.0,    #55
    1007.0: 2007.0,    #07
    1007.0: 2007.0,    #57
    1008.0: 2008.0,    #08
    1009.0: 2009.0,    #59
    1010.0: 2010.0,    #10
    1015.0: 2015.0,    #65
    1016.0: 2016.0,    #66
    1017.0: 2017.0,    #17
    1018.0: 2018.0,    #68
    1063.0: 2013.0,    #63
    1515.0: 2015.0,    #65

}

fix_error_year_dictionary   # print dictionary
```

```
car_df['year_of_registration'].replace(fix_error_year_dictionary, inplace=True)
```

Hence, we have achieved our success in cleaning year of registration column.

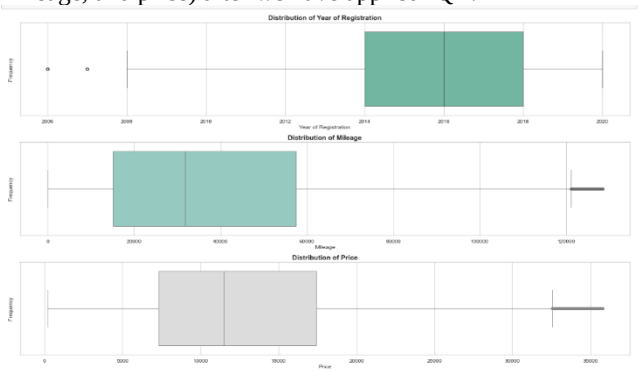
Interquartile Range (IQR) method for outliers:

- Our aim is to remove outliers from numeric columns using the Interquartile Range (IQR) method, which defines outliers as values that:

  Lies below (Q1 - 1.5 * IQR) or Above (Q3 + 1.5 * IQR)

- This ensures that extreme values do not skew analyses or distort patterns in the data.
- Why we do this?: Outliers can heavily influence statistical measures (like mean and standard deviation) and degrade model performance in machine learning. Removing them improves data quality and model accuracy.

First, we checked the shape of our dataset before handling the outliers, since now we have cleaned the data and filled missing values. Now the shape of our dataset is (370426, 9). Then we applied IQR on numeric columns and printed the number of values that we get after outliers has been fixed -Original dataset shape: (370426, 9) Cleaned dataset shape: (332098, 9) Lets check the output via visualization (year_of_registration, mileage, and price) after we have applied IQR:



Comparing this plot with the previous plot, we undermine that our outlier in dataset has been handled well, leading to a more stable and noise free dataset for model training and testing.

```
car_df_cleaned.describe()
```

	mileage	year_of_registration	price
count	332098.000000	332098.000000	332098.000000
mean	38945.724913	2015.467720	12963.786503
std	29738.350521	3.320114	7520.981315
min	0.000000	2006.000000	200.000000
25%	15129.250000	2014.000000	7295.000000
50%	31758.000000	2016.000000	11500.000000
75%	57525.750000	2018.000000	17399.000000
max	128300.000000	2020.000000	35750.000000

Checking the columns in the cleaned dataset on how it looks after the IQR has been performed.

```
# checking each columns to see if now the outliers have been handled:

# Price
car_df_cleaned.sort_values('price', ascending=False).head(3)
```

	mileage	standard_colour	standard_make	standard_model	year_of_registration	price	body_type	crossover_car_and_van	fuel_type
46193	33931.0	Grey	Mercedes-Benz	E Class	2017.0	35750	Saloon	False	Petrol
189775	24075.0	Silver	Porsche	Cayenne	2016.0	35750	SUV	False	Diesel
137481	26869.0	Grey	Porsche	Macan	2017.0	35750	SUV	False	Petrol

```
# year_of_registration
car_df_cleaned.sort_values('year_of_registration', ascending=False).head(3)
```

	mileage	standard_colour	standard_make	standard_model	year_of_registration	price	body_type	crossover_car_and_van	fuel_type
112454	3566.0	Black	Volkswagen	Tiguan	2020.0	32220	SUV	False	Diesel
112460	3000.0	Grey	Toyota	C-HR	2020.0	25190	SUV	False	Petrol Hybrid
112480	3997.0	Silver	SEAT	Tarraco	2020.0	23890	SUV	False	Diesel

```
# mileage
car_df_cleaned.sort_values('mileage', ascending=False).head(3)
```

	mileage	standard_colour	standard_make	standard_model	year_of_registration	price	body_type	crossover_car_and_van	fuel_type
240780	128300.0	Silver	Saab	9-3	2007.0	1595	Saloon	False	Diesel
278158	128300.0	Grey	Audi	A4	2007.0	2300	Saloon	False	Diesel
357621	128300.0	Silver	Vauxhall	Astra	2012.0	2450	Hatchback	False	Diesel

Now checking the output, we conclude that we have successfully cleaned our data frame and handled the outliers. Now our data is ready for further steps.

Data Manipulation (car_df_cleaned): Based on our analysis and understanding, the following data manipulation transformations can be effectively applied to enhance the dataset:

1. **Calculate Vehicle Age:** Derive the age of each vehicle by subtracting the year of registration from the current year. This will basically help in using the Predictive Analysis in future as well.
2. **Categorize Year of Registration:** Transform the year of registration column into categorical variables, grouping vehicles into meaningful time periods. This will clarify the yearly data more specifically in terms of category and supporting price predictive analysis.
3. **Categorize Mileage:** Create mileage-based categories to group vehicles into ranges (e.g., low, medium, high mileage). This will help in segregating the cars in different categories for predictive analysis.

These transformations provide additional insights and simplify analysis by creating structured, interpretable variables.

Step1: Calculated age of vehicle based on current date and year of registration. We imported datetime package and took out current year and passed to a parameter.

```
# creating a new column 'age_of_vehicle' in dataset
car_df_cleaned = car_df_cleaned.assign(age_of_vehicle =  current_year - car_df['year_of_registration'])
car_df_cleaned['age_of_vehicle'] = car_df_cleaned['age_of_vehicle'].astype('int64')
car_df_cleaned.head()
```

	mileage	standard_colour	standard_make	standard_model	year_of_registration	price	body_type	crossover_car_and_van	fuel_type	age_of_vehicle
1	108230.0	Blue	Jaguar	XF	2011	7000	Saloon	False	Diesel	13
2	7800.0	Grey	SKODA	Yeti	2017	14000	SUV	False	Petrol	7
3	45000.0	Brown	Vauxhall	Mokka	2016	7995	Hatchback	False	Diesel	8
4	64000.0	Grey	Land Rover	Range Rover Sport	2015	26995	SUV	False	Diesel	9
5	16000.0	Blue	Audi	S5	2017	29000	Convertible	False	Petrol	7

Step2: Creating a categorical column from year of registration feature.

```
car_df_cleaned.year_of_registration.describe()
```

```
count    332098.000000
mean       2015.467720
std           3.320114
min        2006.000000
25%        2014.000000
50%        2016.000000
75%        2018.000000
max        2020.000000
Name: year_of_registration, dtype: float64
```

To determine bins for categorizing year_of_registration, we need to consider the range, distribution, and the context of data.

Understanding the Data:

- Range: The data spans from 2006 to 2020.
- Median (50%): 2016: This is a good split point for modern vs. older registrations.
- Quartiles (25% and 75%): 2014 and 2018 indicate natural divisions in the data.
- Standard Deviation (std): ~3.32 years indicates moderate spread around the mean (2015.47).
- Based on the data, we can use bins like: Old vehicles: 2006 to 2013 (below the 25% mark). Moderate-aged vehicles: 2014 to 2017 (between 25% and the median, and slightly beyond). New vehicles: 2018 to 2020 (75% onward).

crossover_car_and_van	fuel_type	age_of_vehicle	condition_of_vehicle
False	Diesel	13	OLD
False	Petrol	7	MID-AGE
False	Diesel	8	MID-AGE
False	Diesel	9	MID-AGE
False	Petrol	7	MID-AGE

Step3: Creating a categorical column from mileage feature

```
car_df_cleaned.mileage.describe()
```

```
count     332098.000000
mean       38945.724913
std        29738.350521
min            0.000000
25%        15129.250000
50%        31758.000000
75%        57525.750000
max       128300.000000
Name: mileage, dtype: float64
```

Based on the provided statistics, we can simplify the binning strategy by dividing the data into three categories that represent distinct ranges of mileage.

- Low Mileage: Vehicles with mileage from 0 to just below the 25th percentile (15,129).
- Average Mileage: Vehicles with mileage from 25th percentile (15,129) to the 75th percentile (57,525).
- High Mileage: Vehicles with mileage from 75th percentile (57,525) to Maximum (128,300).

This ensures that each category corresponds to a different range of mileage, and the bins reflect the data distribution with three categories as required. According to the logic above we have created a new column “usage of vehicles”. We have created some categorical columns using feature engineering to utilise it as a bucket to segregate cars based on different conditions which can help in effective price prediction.

fuel_type	age_of_vehicle	condition_of_vehicle	usage_of_vehicle
Diesel	13	OLD	HIGH
Petrol	7	MID-AGE	LOW
Diesel	8	MID-AGE	AVERAGE
Diesel	9	MID-AGE	HIGH
Petrol	7	MID-AGE	AVERAGE

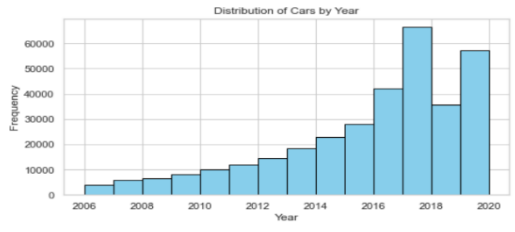
We have successfully transformed data into our required usage. Which can be now utilized for ML-model training and testing. But before that lets perform some EDA.

EDA - Exploratory Data Analysis on Curated Dataframe:

We have now cleaned the data, before delving into model building. We are now plotting to get insights.

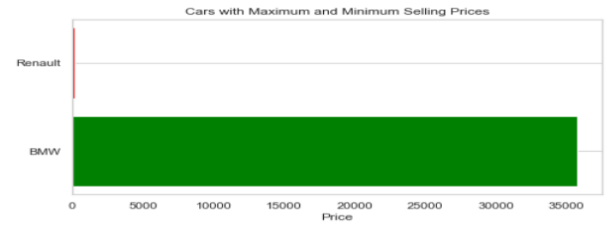
1. Range of Years: We plot the distribution of car registration years to identify trends or anomalies.

```
Range of years: 2006 to 2020
```

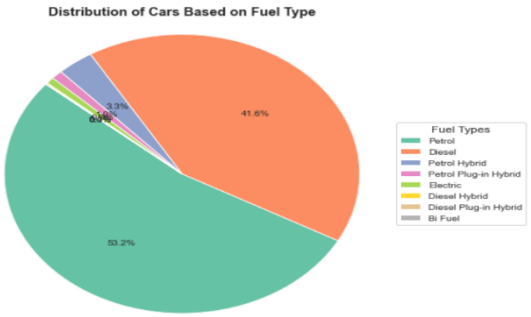


2. Max vs. Min Selling Prices: We plot the highs and lows in selling price based on car mo1del as an exploration.

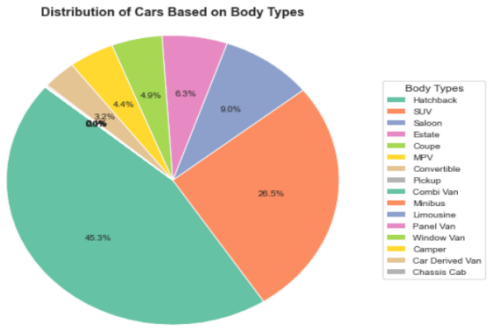
```
Maximum selling price: 35750
Minimum selling price: 200
Car with Maximum Selling Price of  BMW  is in 35750
Car with Minimum Selling Priceof  Renault  is in 200
```



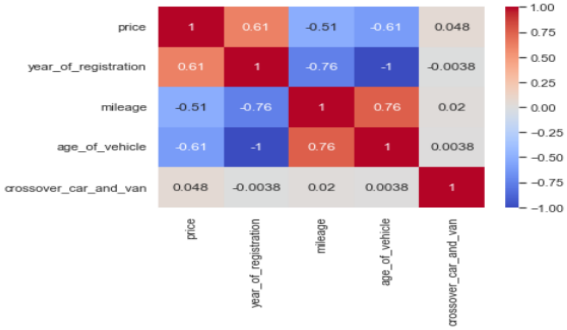
3. **Fuel type:** Following plot display pie distribution of cars based on fuel type.



4. Body Type: Following plot display pie distribution of cars based on body type.



Now, Let's find Correlation between the price and other features.



Correlation Results:¶

- **Price vs. Year of Registration**: A correlation of 0.61 indicates a strong positive relationship.
- **Price vs. Mileage**: A correlation of -0.51 shows a moderate negative relationship.
- **Price vs. Age of Vehicle**: A correlation of -0.61 suggests a strong negative relationship.
- **Price vs. Crossover Car and Van**: A correlation of 0.05 indicates a very weak positive relationship, meaning that the type of vehicle (crossover, car, or van) has minimal influence on the price.

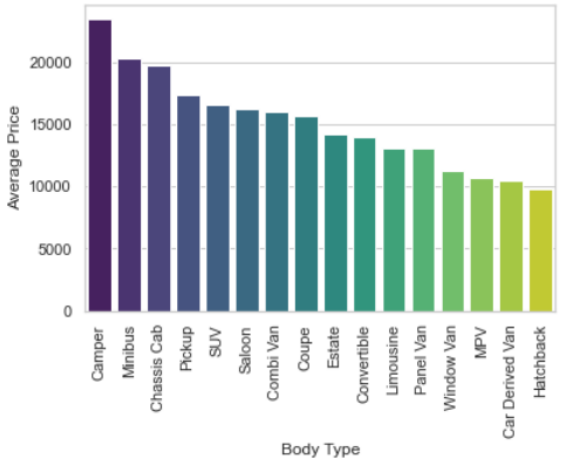
These insights highlight the most significant predictors of price, with age and year of registration being more impactful compared to mileage and vehicle type.

- We have created various plot indicating relationship of price with different features such as Mileage, Year of registration, Age of Vehicle, Crossover Type. This indicates how these features in their contribution to price valuation.

5. Average Price of vehicles based on body type

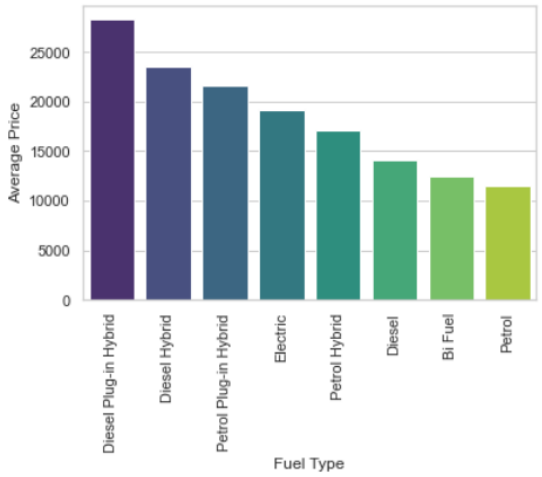
```
# lets get value counts of each body type:
car_df_cleaned.body_type.value_counts()
```

```
body_type
Hatchback          150465
SUV                 87954
Saloon              29962
Estate              21036
Coupe               16315
MPV                 14641
Convertible         10787
Pickup                444
Combi Van             190
Minibus               131
Limousine              62
Panel Van              40
Window Van             40
Camper                 29
Car Derived Van         1
Chassis Cab             1
Name: count, dtype: int64
```



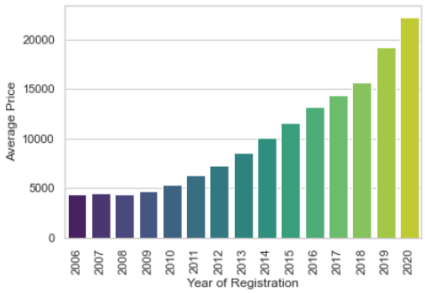
The most common body types are Hatchback, SUV, and Saloon, while Camper, Minibus, and Chassis Cab have the highest average prices, suggesting they are more expensive vehicles. Hatchback has the lowest average price, indicating it is more affordable. These insights can help guide pricing and marketing strategies.

6. Average Price of vehicles based on Fuel type



Cars with Diesel Plug-in Hybrid fuel type have the highest average price, followed by Diesel Hybrid and Petrol Plug-in Hybrid. Electric and Petrol Hybrid cars also have relatively high prices, while Petrol and Bi-Fuel cars are the most affordable. This suggests that higher-priced fuel types may appeal to buyers prioritizing efficiency or eco-friendliness, whereas lower-priced options cater to affordability and convenience.

7. Average Price of vehicles based on year of registration

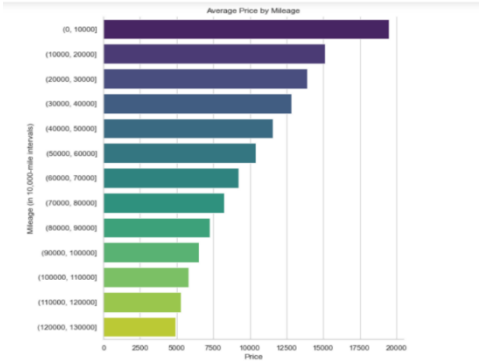


8. Price Trends by Year

The average price of cars decreases with earlier years of registration, with 2020 having the highest average price of 22,319 and 2008–2007 among the lowest, around 4,400. This trend reflects depreciation over time, influenced by wear, market demand, and newer models. Year of registration is a key factor in determining car value, aiding dealerships and sellers in setting fair prices.

14. Average Price of vehicles based on Mileage

Here we checked prices based on mileage – lowest and highest. Next, grouped the data by 10,000-mile intervals to calculate the mean price for each interval and see result.



IV. Data Transformation for Model Training.

we save our cleaned dataset here as csv (locally) inorder to utilize for machine learning purpose - model training.

1. Imported libraries that are required for Model Building.
2. Load cleaned dataset which we transformed for prediction.

- **Label Encoder for transforming Categorical Data**

In the data preprocessing stage, we use LabelEncoder to transform categorical data into a numerical format suitable for machine learning. This transformation ensures compatibility with machine learning algorithms, which require numerical input.

```
# Encoding categorical columns using LabelEncoder
label_encoders = {}

categorical_columns = ['standard_colour', 'standard_make', 'standard_model', 'body_type',
                       'fuel_type', 'condition_of_vehicle', 'usage_of_vehicle']

for col in categorical_columns:
    le = LabelEncoder()
    car_df[col] = le.fit_transform(car_df[col])
    label_encoders[col] = le  # Store label encoder for potential future use
```

```
# Define input and output data (X and Y):
input_data = car_df[["mileage", "standard_colour", "standard_make", "standard_model", "year_of_registration",
                     "body_type", "crossover_car_and_van", "fuel_type", "age_of_vehicle", "condition_of_vehicle",
                     "usage_of_vehicle"]]
output_data = car_df["price"]
```

- **Standardize Numerical Data using StandardScaler**

After Label Encoding, all data is in numeric format. The next step is to standardize the numeric data to the same scale using StandardScaler for uniform comparison and improved model performance.

```
ss = StandardScaler()
input_data = pd.DataFrame(ss.fit_transform(input_data), columns=input_data.columns)

input_data.head(5)
```

The input data has been successfully standardized using StandardScaler. This process ensures that all features are on a similar scale, with a mean of 0 and a standard deviation of 1, allowing for consistent and easier comparisons. Standardization enhances the efficiency and performance of machine learning models, particularly those sensitive to feature scaling. By standardizing the data, our models can deliver more accurate car price predictions, supporting better decision-making in the automotive industry.

	mileage	standard_colour	standard_make	standard_model	year_of_registration	body_type	crossover_car_and_van	fuel_type	age_of_vehicle
0	2.329799	-1.037939	-0.457240	1.698092	-1.345655	1.524517	-0.058379	-1.171532	1.345655
1	-1.047327	-0.273509	0.805764	1.741568	0.461515	1.216141	-0.058379	0.799071	-0.461515
2	0.203585	-0.783129	1.184665	0.519897	0.160320	-0.634114	-0.058379	-1.171532	-0.160320

Model Building and Training

Step1: We split the dataset into input data (X) and output data (Y), where X contains features like car details(colour, model, make, mileage, etc) and Y contains the target variable(Price). This separation allows us to train machine learning models to predict car prices accurately.

Data Partition: Train and Test: We use data partitioning to separate our dataset into training and testing sets. This helps train the model on one set and evaluate its performance on unseen data for reliable predictions. Splitting the data into 80% training and 20% testing.

```
# Split the dataset into training, validation, and test sets
x_train, x_temp, y_train, y_temp = train_test_split(input_data, output_data, test_size=0.2, random_state=42)
x_val, x_test, y_val, y_test = train_test_split(x_temp, y_temp, test_size=0.5, random_state=42)

print("Training set shape:", x_train.shape, y_train.shape)
print("Validation set shape:", x_val.shape, y_val.shape)
print("Testing set shape:", x_test.shape, y_test.shape)
```

```
Training set shape: (265678, 11) (265678,)
Validation set shape: (33210, 11) (33210,)
Testing set shape: (33210, 11) (33210,)
```

We split our standardized dataset into three portions: a training set, validation set and a testing set. The training set, comprising 80% of the data (265678 samples), is used to train our machine learning models. The remaining 20% (33210 samples) forms the testing set and validation set (each 10%), reserved for evaluating the model's performance. Since our target variable represents continuous data, we employed regression techniques for prediction.

For model-building phase, we will explore four approaches:

- Linear Regression
- Decision Tree Regressor
- KNN Regressor
- Random Forest Regressor

Each model was trained using the training dataset and evaluated on both training and testing datasets. To evaluate performance, we used metrics like Root Mean Squared Error (RMSE) and R-squared values. This analysis allowed us to identify the most effective method for accurately predicting car prices, contributing to informed decision-making.

Cross-validation:

- Validation and Test Sets: These should not be used during cross-validation. Cross-validation is exclusively for evaluating model performance on the training data.
- Scoring Metrics: neg_root_mean_squared_error is used for RMSE. The negative sign is negated to calculate RMSE correctly. r2 provides the R-squared score directly.
- Training Set Only for Cross-Validation: The x_train and y_train subsets are used for both model training and validation during the cross-validation process.
- Why Split Before Cross-Validation? The split ensures that the validation and testing sets remain untouched during model training, allowing for an unbiased evaluation of the final model on unseen data.

Hyper-Tuning the model for best result: For hyperparameter tuning using GridSearchCV, we can choose a different scoring metric from the one used in cross-validation, based on your goal:

- Minimize RMSE: Use scoring = 'neg_root_mean_squared_error' to tune the model for low error. (typical for regression problems where you want low errors))
- Maximize R^2: Use scoring = 'r2' to tune based on model fit and variance explained.

Key Points:

- Tuning: Choose a metric that aligns with our goal (RMSE for minimizing error, R^2 for model fit).
- Final Evaluation: After tuning, we can assess the model with RMSE or R^2, depending on your focus.

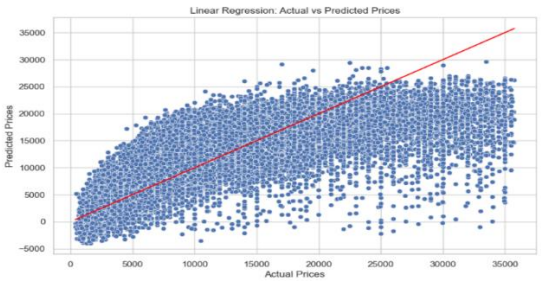
This approach ensures you select the most appropriate metric for both tuning and evaluation.

Model Training:

In this code, we trained all the models on the given training dataset and evaluated its performance using cross-validation to calculate metrics such as RMSE and R^2. We then predicted outcomes for both the training and validation datasets and computed detailed performance metrics, including RMSE, R^2, MAE, and MSE, to assess the model's accuracy and reliability. To visualize the model's effectiveness, we plotted a comparison of actual versus predicted values.

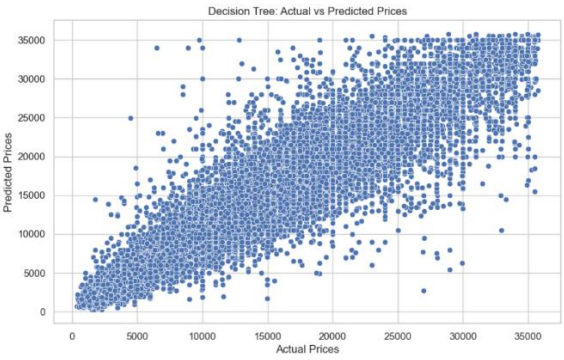
Linear Regression

```
Training Linear Regression...
Cross-validation for Linear Regression - RMSE: 28080315.40151464, R^2: 50.39354319510348
Linear Regression:
    Train RMSE: 5298.7947391519665, Validation RMSE: 5317.10783943165
    Train R^2: 50.40054139519472, Validation R^2: 49.73982589484392
    Train MAE: 4001.976654127086, Validation MAE: 4001.4982578986724
    Train MSE: 28077225.687664554, Validation MSE: 28271635.776145514
```



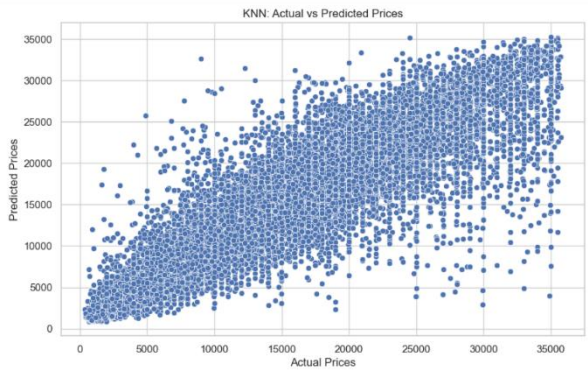
Decision Tree Regressor

```
Training Decision Tree...
Cross-validation for Decision Tree - RMSE: 7174220.621925587, R^2: 87.32555578366103
Random Forest:
    Train RMSE: 284.82526710838056, Validation RMSE: 2643.0844610440004
    Train R^2: 99.85668891969968, Validation R^2: 87.58075672405828
    Train MAE: 39.60663054361109, Validation MAE: 1726.5101336483663
    Train MSE: 81125.43278336033, Validation MSE: 6985895.468212253
```



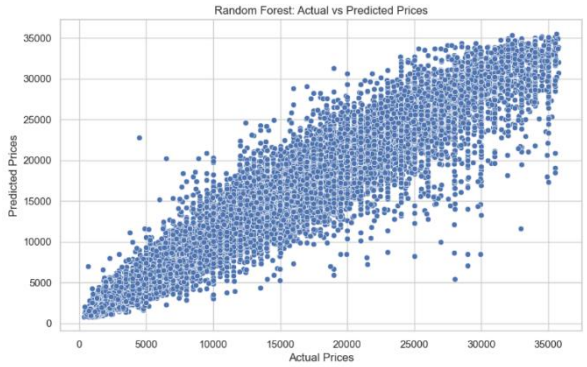
KNN

```
Training KNN...
Cross-validation for KNN - RMSE: 9238107.429820443, R^2: 83.68044795447854
KNN:
    Train RMSE: 2375.5207119465467, Validation RMSE: 3002.1905850513212
    Train R^2: 90.03125732042423, Validation R^2: 83.9767883672762
    Train MAE: 1503.4329203020202, Validation MAE: 1889.4765251430292
    Train MSE: 5643098.652887029, Validation MSE: 9013148.308970794
```



Random Forest Regressor:

```
Training Random Forest...
Cross-validation for Random Forest - RMSE: 4442084.824169274, R^2: 82.19242086170573
Random Forest:
    Train RMSE: 815.60120625225, Validation RMSE: 2076.0495189125713
    Train R^2: 98.82489016261754, Validation R^2: 92.3378884910881
    Train MAE: 532.9246350688876, Validation MAE: 1378.3690850444355
    Train MSE: 665205.3276401252, Validation MSE: 4309981.604977119
```




```
# Hyperparameter Tuning using Random Search - best model
rf_params = {
    'n_estimators': [100, 200, 300, 400], # Number of trees in random forest
    'max_depth': [10, 20, 30, 40],  # Maximum number of levels in tree
    'min_samples_split': [2, 5, 10], # Number of samples
    'min_samples_leaf': [1, 2, 4],
    'bootstrap': [True, False]
}
rf_random = RandomizedSearchCV(rf, rf_params, n_iter=20, cv=5, scoring='neg_mean_squared_error', random_state=42)
rf_random.fit(x_train[:5000], y_train[:5000])

print(f'Best Random Forest Params: {rf_random.best_params_}')
```

```
Best Random Forest Params: {'n_estimators': 300, 'min_samples_split': 5, 'min_samples_leaf': 1, 'max_depth': 20, 'bootstrap': True}
```

In this above code, we trained and evaluated a Random Forest Regressor, which demonstrated superior accuracy compared to other models such as Linear Regression, Decision Tree, and K-Nearest Neighbors. With metrics like RMSE and R^2 confirming its robust performance, Random Forest outperformed the alternatives in both predictive accuracy and generalization. To further refine the model and achieve even better results, we performed hyperparameter tuning using RandomizedSearchCV. This allowed us to identify the best combination of parameters, such as the number of estimators, tree depth, and minimum samples per split, ensuring the Random Forest model delivers optimal performance for our analysis.

Next, we compared RMSE, MAE/MSE and R square and find the best model.

Model Performance Summary:

```
Model Comparison Table:
              Model         RMSE         R²          MAE           MSE
0  Linear Regression  5317.107839  49.739826  4001.498258  2.827164e+07
1      Decision Tree  2643.084461  87.580757  1726.510134  6.985895e+06
2      KNN Regressor  3002.190585  83.976788  1889.476525  9.013148e+06
3      Random Forest  2076.049519  92.337888  1378.369085  4.309982e+06

Best Model Summary:
Best Model: Random Forest
MSE: 4309981.604977119
MAE: 1378.3690850444355
R²: 92.3378884910881
```

- **Linear Regression**: The model exhibited modest predictive capabilities with moderate explanatory power.
- **K-Nearest Neighbors (KNN)**: Achieved a good balance between training and validation performance, reflecting strong predictive ability.
- **Decision Tree Regressor**: Demonstrated excellent predictive accuracy with lower error rates compared to simpler models.
- **Random Forest Regressor**: Outperformed other models, showcasing exceptional predictive power and generalization capability.

Best Model Summary:

- **Best Model**: Random Forest Regressor
- **Metrics**:
  - RMSE: 2076.05
  - R^2: 92.34%
  - MAE: 1378.37
  - MSE: (4.31 times 10^6)

Random Forest consistently outperformed other models, making it the most accurate and reliable choice for this dataset.

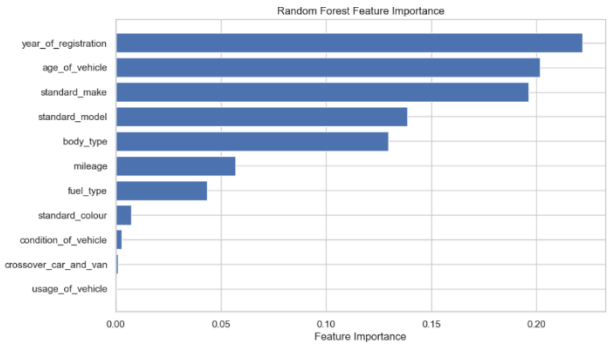
V. Model Evaluation and Analysis

Final Training and Testing (test-set) on Best Model - Random Forest:

Now, going ahead as we have received our best model for predicting car price, as analysed above which is Random Forest model along with the best estimators for the model, which are: 'n_estimators': 300, 'min_samples_split': 5, 'min_samples_leaf': 1, 'max_depth': 20, 'bootstrap': True. We train the best model on our training data and run the cross-validation before running the model for actual predictions on test data.

```
Training Best Random Forest Regressor Model...
Cross-validation for Best Tuned Random Forest Model - RMSE: 4073959.2315034596, R^2: 82.19242086170573
Best Tuned Random Forest Model Evaluation:
    RMSE: 2003.9944311875624, R^2: 92.89622813507833, MAE: 1333.9394091665613, MSE: 4015993.6802307623
    Training Time: 2886.047844648361 seconds
    Prediction Time: 10.714682817459106 seconds
```

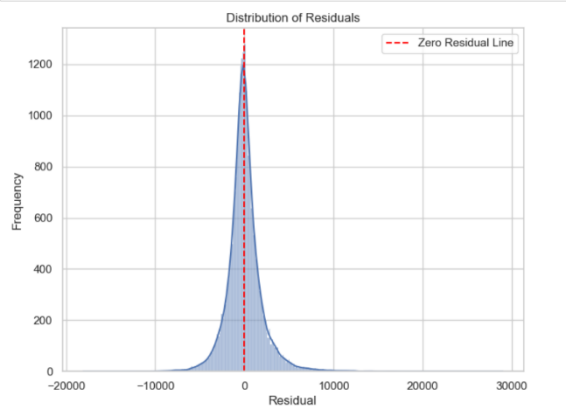
Feature Importance: Identify which features contribute most to your model's predictions



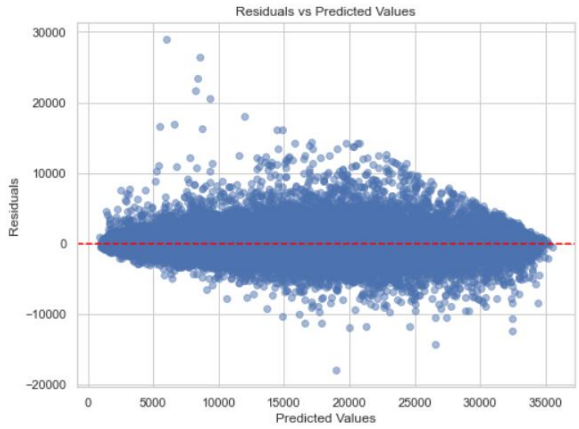
We see that as per our analysis the feature which contributes to the most towards price of car is year of registration which can be understood also from our previous exploration, for example: the car with oldest year of registration is cheaper compared to the car with newest year of registration. Followed by make and model of car, which can be as per users choice but it equally contributes to the car price valuation. We can also see that mileage also becomes an effective parameter which defines price as well.

This helps us understand which features are driving your model's predictions and can guide future feature engineering.

Residual Analysis: Analyse the residuals (errors) to identify patterns that the model might not have captured:



Residual analysis shows discrepancies at extreme values, indicating challenges in modelling complex interactions. Improvements include: Enhancing features via engineering and combining weak ones. Using advanced algorithms like XGBoost and LightGBM. Applying aggressive cross-validation and stratified sampling. Handling outliers with techniques like Winsorization. These steps will improve model accuracy, generalization, and handling of extreme values.



Histogram of Residuals: Should ideally follow a normal distribution centered around zero.
Residuals vs Predicted Values: Random scatter indicates a good fit. Patterns may suggest model limitations or data issues.

Model Prediction:

In the final analysis, we are testing our model to see the level of accurate predictions our model can make.

For this, we are first loading our saved trained best random forest model saved as pickle file, along with saved encoders and standard scaler. We perform 3 types of prediction testing which can be seen in our code notebook.

Type 1: Prediction with older data

```
Predicted Selling Price: 1714.665356743179
```

We use a trained Random Forest model to predict car prices by encoding categorical variables, standardizing numerical data, and preparing inputs for prediction. In this example, the model predicts a price of 1685.64 Pounds, close to the actual price of 2000 Pounds, demonstrating its accuracy in aiding informed decisions for car buyers and sellers.

Type 2: Prediction with New data

This helps in understanding how our model will react in case of new data provided to it, using which it is never trained on, as it becomes unseen data for model.

```
Predicted Selling Price: 18396.1535323684
```

Type 3: User Input Data Prediction

This code allows users to input car details like name, year, price, mileage, and fuel type to predict the selling price using a trained Random Forest model. It processes categorical data, standardizes inputs, and predicts values.

```
Enter mileage: 45000.0
Enter car color: Brown
Enter car make: Vauxhall
Enter car model: Mokka
Enter year of registration: 2016
Enter body type: Hatchback
Is it a crossover (1 for Yes, 0 for No): 0
Enter fuel type: Diesel
Predicted Selling Price: 9062.688504755071
```

Conclusion:

Understanding about the dataset is a critical component of top-notch models because without proper understanding of the data, any further modeling cannot be successfully carried out. In this step, we carefully examined the structure, quality and the subtleties of the data because of exhaustive searches. This involved finding the locations of these features, knowing their characteristics and finding out how the variables related to each other.

These insights were also very helpful in the feature selection and engineering domain. We also discussed issues that could come up when working with the data such as missing values, outliers and skewed distribution to pre-process the data for applying models in machine learning. With the help of these findings, we applied machine learning algorithms to develop models that may discover such patterns and generate accurate predictions. But more than refining the achievement of prediction, these models facilitated constructive, data-driven decisions; thereby enriching the analytic worth and promoting the correct achievement of strategies determinately.

Future scope:

- **Integration of Advanced Models**: Explore more sophisticated models like Gradient Boosting or Neural Networks for potential improvement.
- **Handling Extremes**: Develop methods to better predict outliers or rare cases in pricing.
- **Data Enrichment**: Incorporate real-time or external data sources like economic indicators or customer preferences.
- **Automation**: Create automated pipelines for continuous data updates and model retraining.
- **Deployment**: Scale the model for live platforms, enabling dynamic pricing solutions.